Convolutional Neural Network (CNN)

PART 1: What is a CNN?

A **Convolutional Neural Network (CNN)** is a neural network **designed for images**.

Why normal neural networks fail for images:

- Image has **too many pixels**
- Fully connected layers → **huge number of weights**
- No idea about **spatial structure** (edges, shapes)

CNN solves this by:

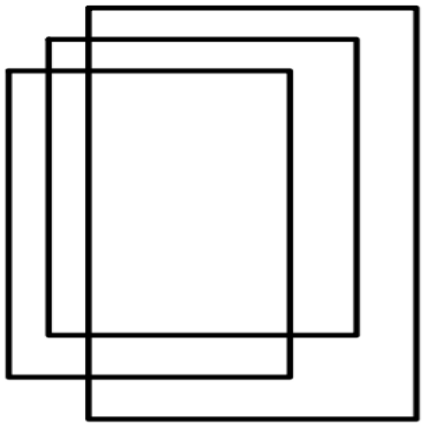
- Looking at **small parts of image at a time**
- Reusing the **same weights** everywhere
- Learning **edges → textures → shapes → objects**

PART 2: How an Image is Represented

Suppose RGB Image = 227 × 227 × 3

Meaning:

- Height = 227
- Width = 227
- Channels = 3 (Red, Green, Blue)



RGB Image 227X227X3

So actually:

- R channel → 227×227
- G channel → 227×227
- B channel → 227×227

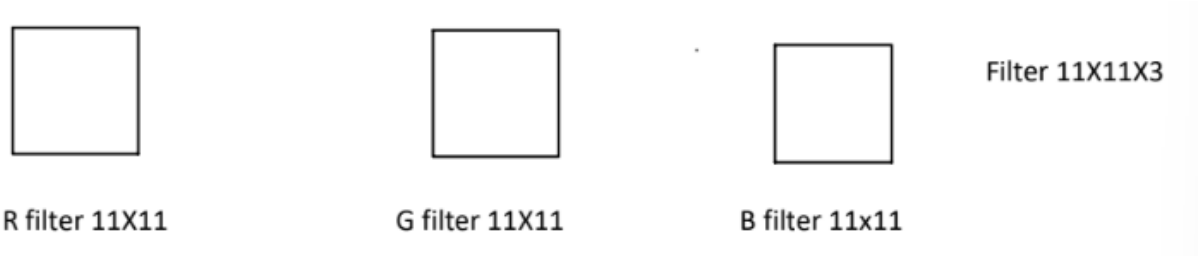
PART 3: What is a Filter (Kernel)?

Suppose Filter size = 11 × 11 × 3

Meaning:

- Filter covers **11×11 pixels**
- Goes through **all 3 channels**

Total values (weights) = 11 × 11 × 3 = 363



💡 Think of a filter as **a small window with numbers** that slides over the image.

PART 4: How Convolution is Actually Calculated

At ONE position:

1. Take **11×11×3 pixels** from image
2. Multiply each pixel with corresponding filter value
3. Add everything

4. Add **bias**
5. Result = **one number**

Formula:

R1F1 + R2F2 + ... + R121F121+

G1F1+G2F2+G3F3+.....+G121F121+

B1F1+B2F2+B3F3+.....+B121F121

This is just **dot product**

So: **Output = Σ(Image values × Filter values)**

PART 5: Sliding the Filter (Stride)

Stride (S) = 4

Padding (P) = 0

Filter moves:

- 4 pixels at a time horizontally
- 4 pixels vertically

PART 6: Output Size Calculation

Formula:

Output size = (W − F + 2P)/S + 1

- W = 227
- F = 11

- P = 0
- S = 4

Calculation:

(227 − 11 + 0)/4 + 1
= 216/4 + 1
= 54 + 1
= 55

Output feature map size = **55 × 55**

PART 7: What is a Feature Map?

- ONE filter → ONE feature map
- Size = **55 × 55**

If we use 90 filters → 90 feature maps

So output becomes: 55 × 55 × 90

PART 8: Number of Weights

For ONE filter:

11 × 11 × 3 = 363 weights

Bias=1

For 90 filters:

Weights = 90 × 11 × 11 × 3

Biases = 90

PART 9: Why Weight Sharing?

Same filter is used everywhere

Meaning:

- Same weights applied across image
- Detects same feature everywhere
- **Reduces parameters**
- Prevents overfitting

PART 10: Convolution as Matrix Multiplication

Converts image into matrix:

- Image → **363 × 3025**
    - 363 = pixels per block
    - 3025 = 55 × 55 positions
- Filters → **90 × 363**

Matrix multiplication:

$(90 \times 363) \times (363 \times 3025) = 90 \times 3025$

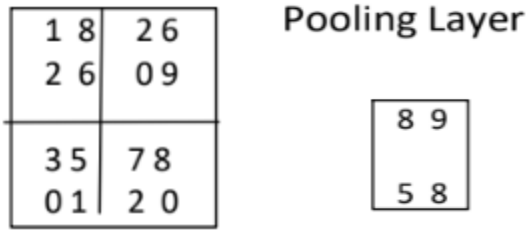
Note: $(A \times B) \cdot (B \times C) = (A \times C)$

Reshape:

$90 \times 55 \times 55$

PART 11: Pooling Layer (Max Pooling)

2 × 2 Max Pooling, Stride = 2



What it does:

- Takes **maximum value** in each 2×2 block
- Reduces size by half
- No weights

Example:

Input: 55 × 55

Output: ~27 × 27

Why pooling?

- Reduces computation
- Adds translation invariance

PART 12: Normalization

Purpose:

- Stabilize training
- Faster convergence
- Prevent exploding/vanishing gradients

Main types (memorize difference):

Type	Normalizes
Batch Norm	Over batch
Layer Norm	Per image
Instance Norm	Per channel
Group Norm	Channel groups

PART 13: VGGNet

Key idea:

- Uses **only 3×3 convolutions**
- Uses **2×2 max pooling**
- Deep but simple

Weight calculation example:

CONV3-64:

(3 × 3 × input_channels) × 64

Example:

(3 × 3 × 3) × 64 = 1,728 weights

Summary:

CNN Pipeline:

Image → Convolution → ReLU → Pooling → (Repeat)
→ Fully Connected → Softmax

Output size formula:

$(W - F + 2P)/S + 1$

Filter weights:

$F \times F \times D$

Feature maps:

Number of filters = Output depth

Pooling:

- No weights
- Reduces size

Q. An image **(201x201x3)** is convolved with **100** filters **(7x7x3)** with stride = **2** and produces **98x98x100** outputs. Represent this convolution operation by **two matrix multiplication.** Explain **what each row and column of the matrices represent**. **How many parameters** will be needed to perform the convolution operation?

Given

- **Input image**: 201×201×3
- **Number of filters (K)**: 100
- **Filter size**: 7×7×3
- **Stride (S)**: 2
- **Output size (given)**: 98×98×100

Part A: Represent convolution as two matrix multiplication

Step 1: Size of one convolution block

Each filter covers:
7×7×3=147 values

So **one sliding window = 147 pixels**

Step 2: Number of sliding positions

Output spatial size: 98×98=9604

So Total convolution positions = **9604**

Matrix Representation

Image Matrix

Shape: 147×9604

What it represents:

- **Each column** → one flattened 7×7×3 image patch
- **Each row** → one fixed pixel position inside the filter window across all patches

So:

- 147 rows = pixel locations
- 9604 columns = all sliding locations

Filter Matrix

Shape: 100 ×147

What it represents:

- **Each row** → one filter (flattened)
- **Each column** → weight corresponding to one pixel position

Matrix Multiplication

$(100\times147)\times(147\times9604)=100 \times 9604$

Output Reshaping

Reshape:
$100\times9604\Rightarrow98\times98\times100$

✔️ Matches given output

Part B: Explain rows and columns clearly

Image Matrix (147 × 9604)

- Each **column** represents one (7×7×3) region of the input image
- Each **row** corresponds to a specific pixel location within the filter window

Filter Matrix (100 × 147)

- Each **row** represents one convolution filter
- Each **column** represents a weight applied to a specific pixel position

Part C: Number of Parameters

Weights per filter:

$7 \times 7 \times 3 = 147$

Total weights:

$100 \times 147 = 14{,}700$

Biases:

- One bias per filter: 100

Total Parameters

14700 weights + 100 Biases =14800 parameters

Prepared By Azmari Sultana✨